



perforce

Game Jam Like the Pros

Practical and free uses for Perforce P4

Jase Lindgren

Principal User Advocate, P4

Perforce Software

Everyone can probably relate to this

Overwritten by a teammate. Lost to a laptop crash. Or tangled in a naming scheme that stopped making sense three files ago. This isn't unique to games, but it hits us extra hard because of how interconnected all the parts of a game are.

- 📁 JamGame/
- 📁 JamGame_v2/
- 📁 JamGame_FINAL/
- 📁 JamGame_backup_DO_NOT_DELETE/
- 📁 **JamGame_final_v2_ACTUALLY_final/**
- 📁 JamGame_final_v2_ACTUALLY_final_USE_THIS_ONE/

Version control isn't corporate process. It's **infrastructure** for anything you want to survive past today.

Whether that's a 48-hour jam game, a personal passion project, or the start of your brand new studio.

WHY THIS MATTERS

This is already table stakes.

94%

of developers surveyed in 2026 now use version control, up from 86% a year earlier.

Build like the pros from day one and you never have to retrofit it later, when it hurts.

37%

of people in real-time industries say moving large files is slow or difficult — the most-cited collaboration problem there is.

Even for a small Godot project, the art assets can add up, and Dropbox or Google drive makes performance and conflicts worse.

94%: Perforce 2026 State of Real-Time Workflows Report, all-industry. 37%: Perforce 2025 State of Game Technology Report, gaming respondents; 36% across all industries.

If this becomes something real.

A co-founder joins. You hire a contractor. You pitch a publisher or an investor.

What they can actually read:

#147	Boss arena lighting pass	mira
#146	Fix double-jump on slopes	sam
#145	Menu stingers, second pass	m theo
#144	Hero texture locked for rework	mira

Four weeks of work, legible in ten seconds — instead of a folder called ACTUALLY_final.

How you've managed the project is part of what makes you look credible. Clean history and a sane project structure is something technical due diligence actually looks at.

A N D A S A B O N U S

These same practices make you a stronger hire. I've seen students get jobs straight out of college specifically because they had professional-level Perforce version control experience.

Most people think version control means **git add, commit, push**.

That's like saying you know how to cook because you can boil water.

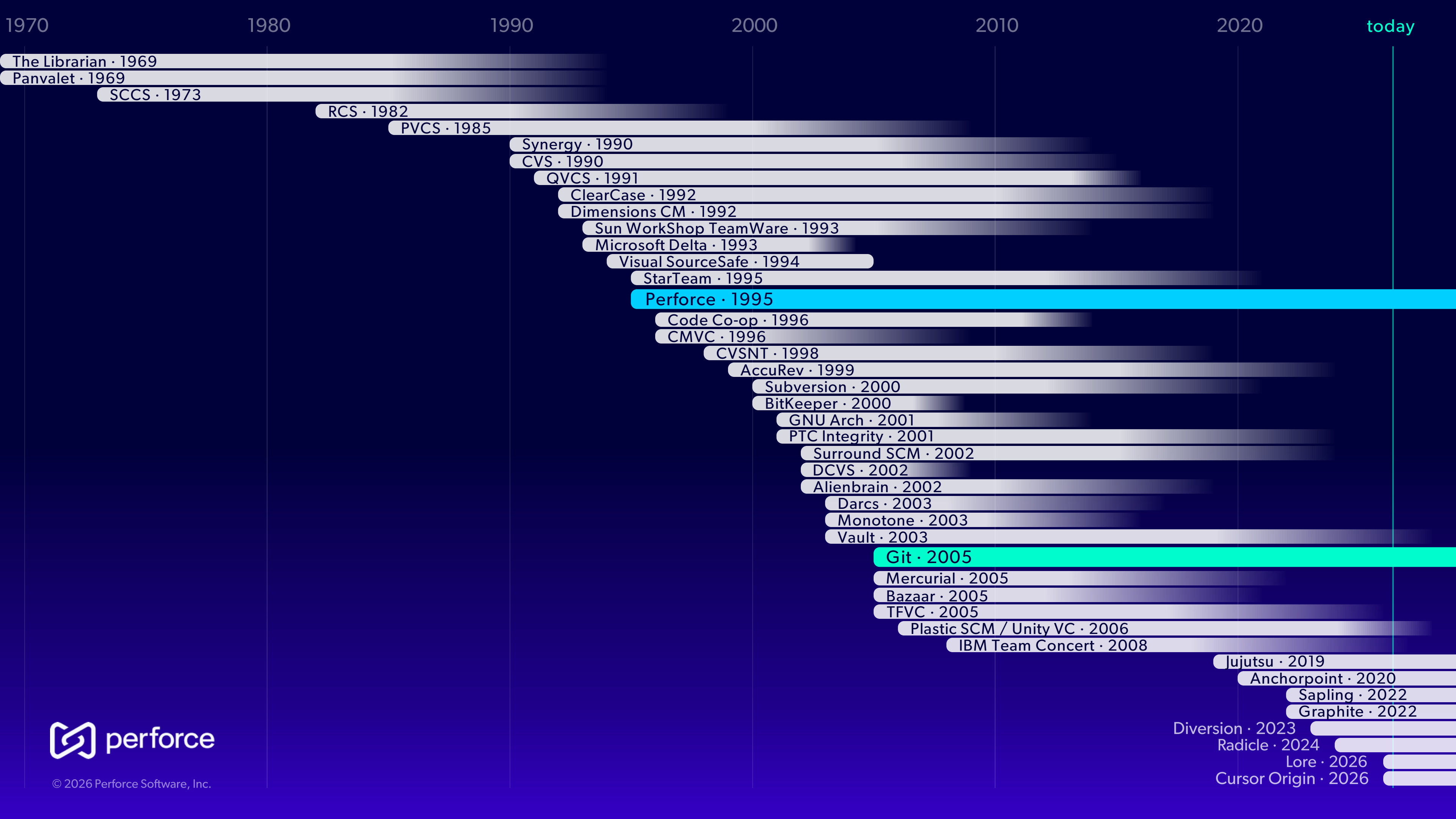
Why different tools exist for different problems.

How your setup has to change as you go from working alone, to a three-person jam team, to a real studio.

The difference between managing code and managing a game project — which means massive binary assets: art, audio, video.



A Brief History of Version Control



Each one was built for a different problem.

Ten years apart, by two people with different constraints.

Christopher Seiwald, 1995

Written in a basement in Alameda, with the first servers in a paint closet.

The problem: a team in one building sharing one large, growing depot, on 1995 hardware. Everyone needed the same answer about what is current, right now.

So: one authoritative server, workspaces instead of copies, and speed as the headline feature. History is stored on the server and cannot be changed. Microsoft licensed it in 1999 to build Windows rather than use its own tool.

Linus Torvalds, 2005

Written in about ten days, after BitKeeper's licence was pulled from the Linux kernel.

The problem: Contributors to Linux each needed their own full copy of history, frequent branching and merging, needed a way to ensure changes to the revision history can be verified since there is no central server.

So: every clone holds the whole history, branching and merging are the everyday operations, each repo can have its own separate history and history can be amended and changed.

Git is not GitHub.

Git is open-source software. The others are products built around it.

Git

Open-source software, GPLv2, 2005. Anyone is free to run it, modify it, host it or build on it. That is part of why many new version control systems are built on top of it.

It defines commits, branches and merges — and nothing about servers, accounts or reviews.

A forge

GitHub, Bitbucket and Azure Repos are commercial services. GitLab ships a community edition; Gitea and Forgejo are open source.

Each adds the central network: a canonical remote, accounts, permissions, pull requests, branch protection. You can host your own.

And P4

A P4 server can also host Git repositories.

Code in Git, assets in the depot, one server behind both.

TWO VOCABULARIES, ONE SET OF IDEAS

Most version control concepts are shared

THE IDEA

Where the history is kept

Your local copy of the files

A set of changes, recorded together

Getting everyone else's changes

Publishing yours

Setting aside changes you don't want to submit yet*

A separate line of development

How you request a review before submitting

IN GIT

repository

clone / working tree

commit

pull

push

stash

branch

branch/fork + pull request*

IN P4

depot

workspace

changelist

sync

submit

shelve*

stream or branch

shelve + review

CONCEPT 1

The repository is the whole story.

Git calls it a repository, P4 calls it a depot. Same job: it keeps every version of every file, along with extra data about who, why, what, and when.

CHANGE	DESCRIPTION	USER	TIME
#1482	Add hairball projectile VFX VFX/fur_trail.tres #1 add Art/FX/collar_diffuse.png #1 add Scripts/laser_pointer_follow.cs #6 edit	mira	10 min ago
#1481	Fix double jump off catnip Scripts/Player/Controller.cs #12 edit	sam	this morning
#1480	Scratching posts in alley Levels/living_room.tscn #9 edit Props/scratching_post.tscn #1 add	theo	yesterday
#1479	New Whiskers idle loop Art/Characters/whiskers_idle.fbx #4 edit	mira	yesterday

...and every version before that, back to the first one.

Save points at every change

You can open any file as it was at any point, and restore it.

Who, when and why

Every submission has an author and a message. Good descriptions give context to the changes.

Single Source of Truth

No separate folders on a laptop, not a zip file in a chat thread, no debate about what is “latest.”

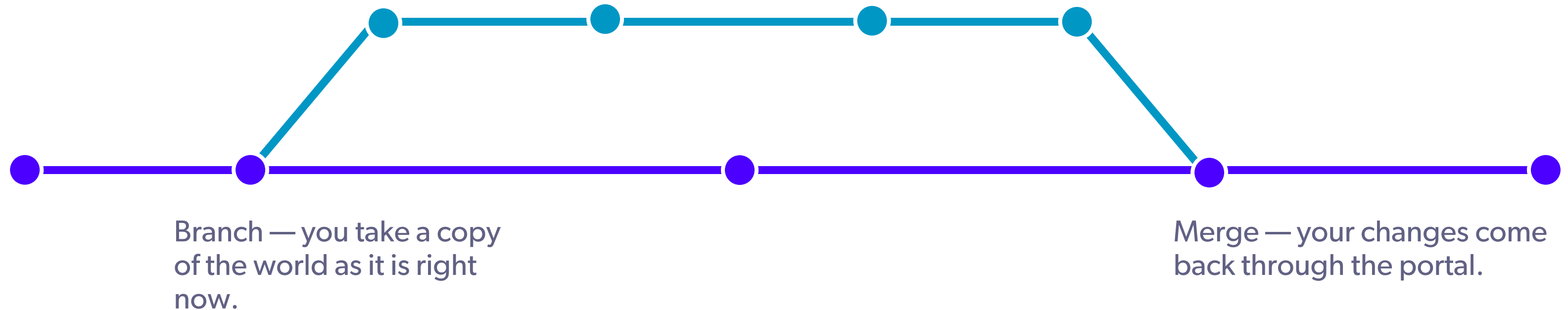
CONCEPT 2

A branch is a parallel universe.

You step sideways into your own copy of the project, work freely, and bring it back when it's ready.

your branch

main



Main keeps working the whole time you're away. And if the idea turns out badly, you throw the universe away instead of trying to pull the changes out.

CONCEPT 3

One change, all or nothing.

That's what atomic means: the whole set lands together, or none of it does.

Changelist #147 — boss arena lighting

arena_boss.mesh

arena_boss.material

BossLights.cs

SUBMIT

The depot

All three files arrive as change #147, in in one step.

If any one of them fails — a lock, a dropped connection, no disk space — nothing lands at all. You fix it and submit again.

A mesh without its material is a broken game.
Nobody ever syncs two files out of three.

And every point in the history is a specific, intentional snapshot in time — so “go back to #146” is a complete change, less hunting for related files.

CONCEPT 4

Where your changes actually live.

Two places in P4. Three in Git — and the middle one catches everybody out.

In P4

Your workspace

Changed on your machine. Nobody else can see it.

SUBMIT

The depot

On the server. Everyone has it on their next sync.

In Git

Working tree

Edited, not recorded yet.

COMMIT

Your local repo

Recorded, still only on your machine.

PUSH

The remote

Shared. Your team can pull it.

This is why work disappears with a dead laptop, and why “I fixed it” sometimes means nobody else can see the fix. Committing is saving. Pushing is sharing.

SAME IDEA, DIFFERENT MECHANICS

Four places they behave differently.

Saving versus sharing

In **Git** it's two steps:

commit records locally, push shares it.

In **P4** a submit does both at once — there is no local-only history (normally)

Unfinished work

A **Git** stash is yours alone.

A **P4** shelf lives on the server, so a teammate can unshelve it, review it, or finish it for you.

(This is also what is used for Code Reviews instead of a forge's Pull Request)

Permissions

In **Git** if you have access to clone the repo you get everything in it.

P4 permissions can be set for specific paths within a depot, so certain groups can be allowed to read or write different files.

What you get locally

A **Git** clone brings the whole repository and its whole history. **P4** workspace syncs only the specific versions of the file paths you ask for — useful when the project is hundreds of gigabytes.

Branching

Git branches are local so they don't exist for anyone else until you push them. They are fast to create and can be merged with any other branches (no rules).

P4 branches and streams are defined on the server for everyone. Streams are set up with rules about what merges where and which should be more stable.

What each one is built for.

Git

BUILT FOR

Working offline — the full history is on your machine.
Branching everything and merging constantly
Contributors you have never met; a forge adds pull requests on top
Open source projects
Code-only projects

P4

BUILT FOR

Binary assets + Code — art, audio, video, engine content.
File locking, enforced by the server
Streams, for branching with guard rails
Importing one project into another, (no submodules required)
Review on a shelved change, so review does not need a branch

P4, Git, or Both?

All in one

Many studios keep code, art assets and much of the pipeline in a single system, and reach for the other only for what they open-source or a small local side project.

One source of truth is what makes builds and pipelines manageable — one place to track a change, one place to coordinate it.

Split by job

Others run both at once and divide by what the files are: the game project and its assets in one, the tools, plugins and shared code in the other.

Splitting the source of truth takes a little extra work to make sure builds always get the right versions of each.

Split by project

And some choose per project — this game in Git, that one in P4, picked based on project needs.

Either way, the transferable skill is the same: how the two differ, where they agree, and the basics of working in each.



What This Looks Like in Practice

OPTION B

A day in the life of a four-person team.

1

Morning sync

Everyone pulls the night's work before touching anything.

2

Feature work

One programmer in code, two artists/designers in the level, one in audio.

3

A binary asset gets locked

An artist checks it out a model. Everyone else can see that. If someone else tries to check it out, they will be stopped to avoid conflicts.

4

A build fails

Automated builds happen with each submit or at a regular interval. Version Control shows you exactly what changed and lets you roll it back if needed.

5

Hotfix at 2am

One file, one changelist, submitted without dragging the rest of the day along.

None of that is process for its own sake. Every stop is somebody not losing an afternoon.

Git CLI

```
sam@git $ git pull
From https://github.com/team/PurrForce
   65779e4..bafcff7  main       -> origin/main
Updating 65779e4..bafcff7
Fast-forward
 levels/living_room.tscn | 6 +++---
 1 file changed, 3 insertions(+), 3 deletions(-)
```

Open file **cat/cat.gd** in Godot/VS Code/etc. and edit it.

```
sam@git $ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be
  committed)
  (use "git restore <file>..." to discard changes in
  working directory)
        modified:   cat/cat.gd

no changes added to commit (use "git add" and/or
"git commit -a")

sam@git $ git add cat/cat.gd

sam@git $ git commit -m "Add pounce"
[main 6c7e1e2] Add pounce
 1 file changed, 3 insertions(+)

sam@git $ git push
To https://github.com/team/PurrForce.git
   bafcff7..6c7e1e2  main -> main
```



P4 CLI

```
sam@p4 $ p4 sync
//purrforce/main/levels/living_room.tscn#2 -
updating C:\PurrForce\levels\living_room.tscn
```

```
sam@p4 $ p4 edit cat/cat.gd
//purrforce/main/cat/cat.gd#1 - opened for edit
```

Open file **cat/cat.gd** in Godot/VS Code/etc. and edit it.

```
sam@p4 $ p4 status
cat\cat.gd - submit change default to edit
//purrforce/main/cat/cat.gd#1
```

```
sam@p4 $ p4 submit -d "Add pounce"
Submitting change 3.
Locking 1 files ...
edit //purrforce/main/cat/cat.gd#2
Change 3 submitted.
```

Large File Handling

Git size limits

Git was designed for text files originally, so it has a small file size limit natively.

- When a repository gets large it becomes very slow
- Forges usually have a size limit per file, too
- Git LFS is an addon that can be added to git
 - LFS is configured to store certain file types in a separate storage location, usually provided by your forge (e.g. github, gitlab, etc)
- Your forge will also have specific storage limits.

Example, GitHub Free tier:

100 MiB per file in Git

2 GB per file in LFS

10 GiB of LFS storage (total of all versions of all files)

```
sam@git $ git push
remote: error: File art/cat_fur.psd is 123.00 MB; this exceeds
GitHub's file size limit of 100.00 MB
remote: error: GH001: Large files detected. You may want to try
Git Large File Storage - https://git-lfs.github.com.
! [remote rejected] main -> main (pre-receive hook declined)
```

```
sam@git $ git push
batch response: This repository exceeded its LFS budget. The
account responsible
for the budget should increase it to restore access.
```

Perforce P4 was built for binary and text from the start, which is why it is most commonly used in gaming, VFX, animation, and other industries that use images, audio, 3D models, CAD files, etc.

File Locking Saves Time

What happens if two people make changes to the same image, model or other binary file?

In Git

Because git is naturally offline-first, a user doesn't know what others are working on.

When the second user tries to push, they get an error.

Nothing can merge two PNGs, so one version is chosen and the other person's work has to be deleted and redone.

In Perforce

Specific file paths can be configured as *exclusive-checkout* on the server

The first check-out locks the file.

The second person is stopped from checking it out until the first user either reverts it back or submits a new version.

This prevents the conflict, *before* you wasted hours of work

This is the most common example of distributed vs centralized version control systems

Automate your builds *before* you even start

Just setting up a manual script you can run from any machine is better than nothing

The most common way a team misses a submission deadline: the build broke hours or days ago and nobody noticed until it's become a big job to fix.

```
p4 sync  
build.bat  
smoke_test.exe  
# does it launch?  
# does the main scene load?
```

A quick, short build script is all it takes.

You can grow into Jenkins, TeamCity or Epic's Horde when you're ready.
You don't need any of that today — you just did it by hand.

Set this up *before* you do a game jam

Godot Skeleton Demo builds //gdc_demo_stream/Godot/Skeleton2D

Build nowRelease build

Release builds start from scratch and make the version players get. They're slower.

Passing: change 3122 by jlindgren

Made the eyes more loving

[Download the newest good build](#) (change 3122, 35.1 MB)

Unzip it, then run the game. Earlier builds are below.

#	Result	Kind	Change	Who	What	Why	Started	Took	Files
6	passed	test	3122	jlindgren	Made the eyes more loving	new change	Tue 22:41	14s	Download 35.1 MB · log
5	passed	test	3121	jlindgren	Adding export config and p4 plugin	new change	Tue 21:30	13s	Download 35.2 MB · log
4	failed	test	3119	jlindgren	P4Godot selftest: cleanup of p4godot_submit_test.txt	button	Tue 21:27	4s	log
3	failed	test	3119	jlindgren	P4Godot selftest: cleanup of p4godot_submit_test.txt	button	Tue 21:24	4s	log
2	failed	test	3119	jlindgren	P4Godot selftest: cleanup of p4godot_submit_test.txt	button	Tue 21:17	4s	log
1	failed	test	3119	jlindgren	P4Godot selftest: cleanup of p4godot_submit_test.txt	new change	Tue 21:15	12s	log



Free and open-source on Github
[jase-perf/p4-build-machine-lite](https://github.com/jase-perf/p4-build-machine-lite)



Getting Started Is Free

Free to get started

P4

Free for up to 5 users and 20 workspaces. Full-featured — the same server AAA studios run.

P4 ONE

The newer client, with a simpler interface that bridges the centralized and local models.

Free for solo use, works with the free server.

Git

Free and open source, with GitHub, GitLab and Bitbucket free for personal use.

Learn it too. You'll need both.

AN HOUR, ON THE MACHINE YOU HAVE

Try it yourself, start to finish.

1

Stand up your own server

You can install on P4 Cloud or just self-host for free on your own computer to get started.

2

Configure it for a game project

Set file types so art and audio lock, then create a depot and a stream. [See the P4 Quickstart for Examples.](#)

3

Create a workspace

Your workspace is the folder where you make changes, before you submit them back to the server.

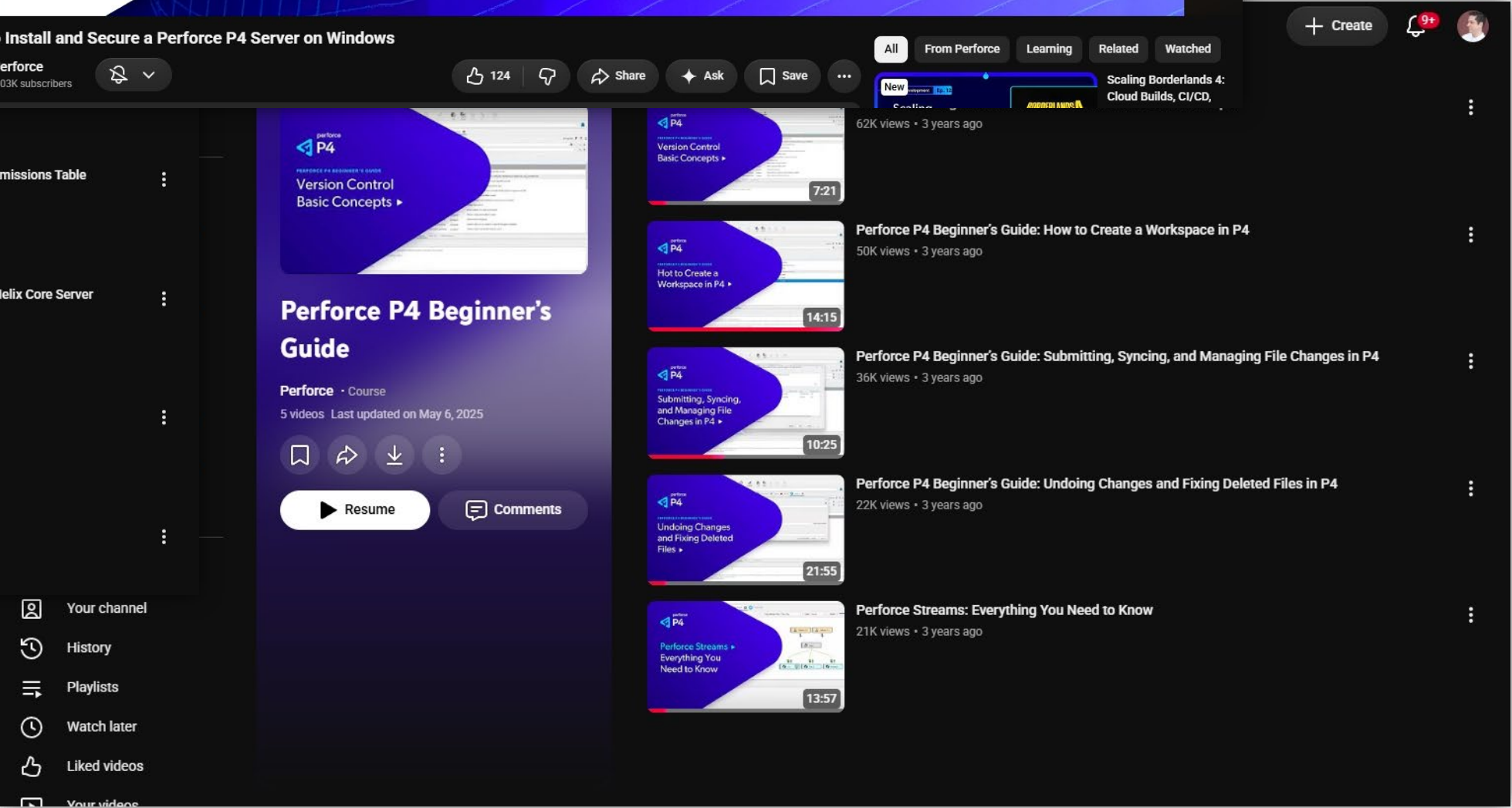
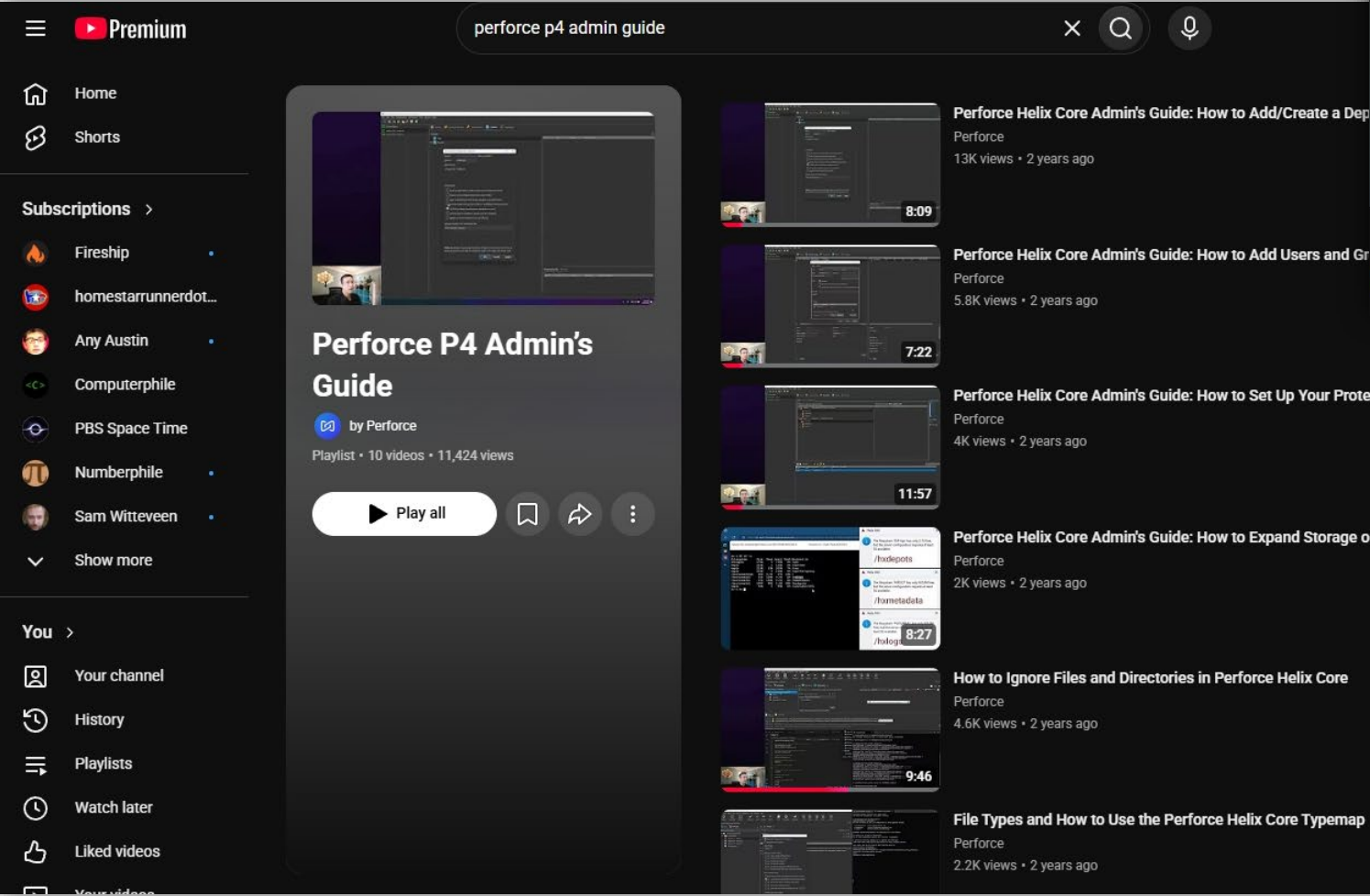
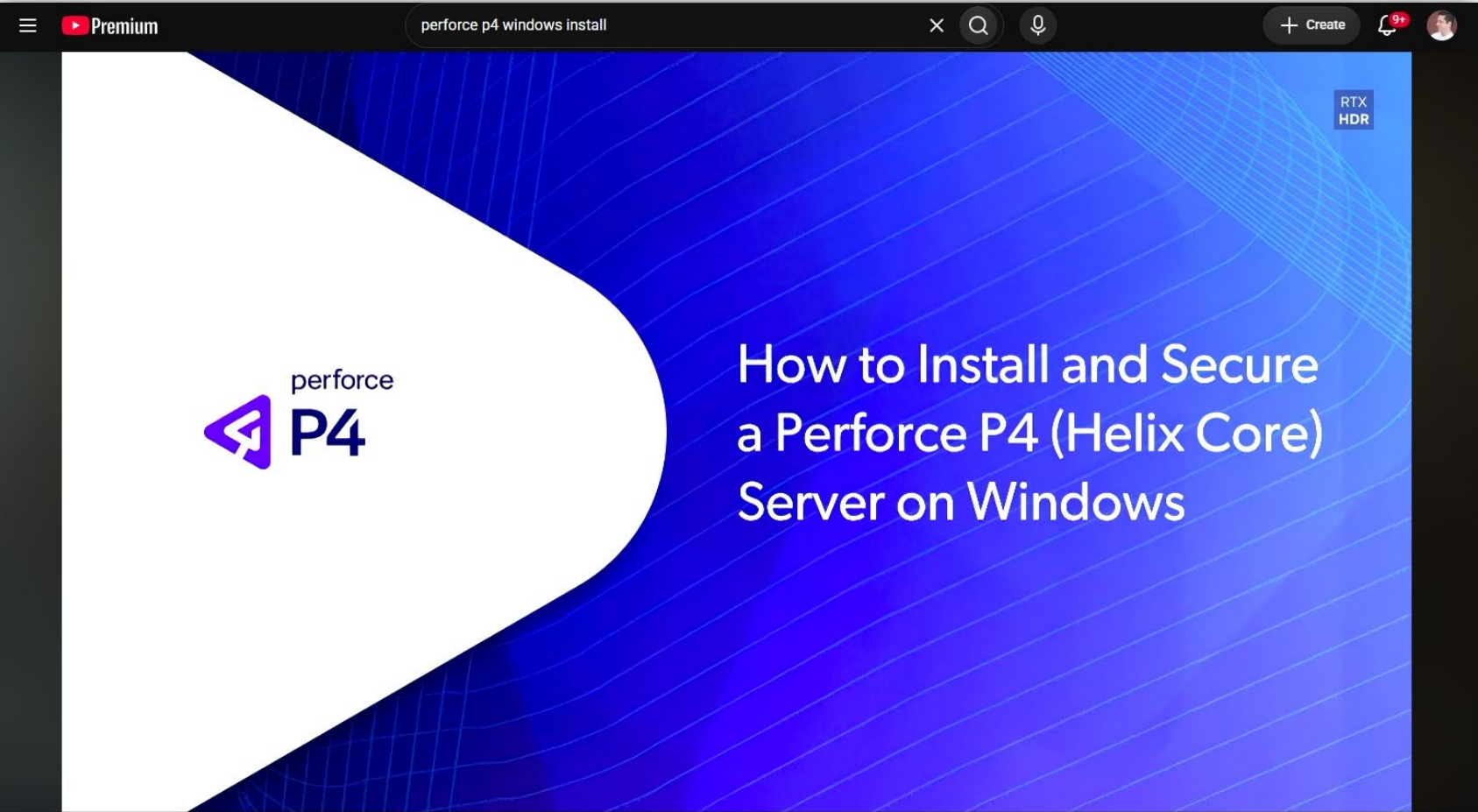
4

Submit, and keep building

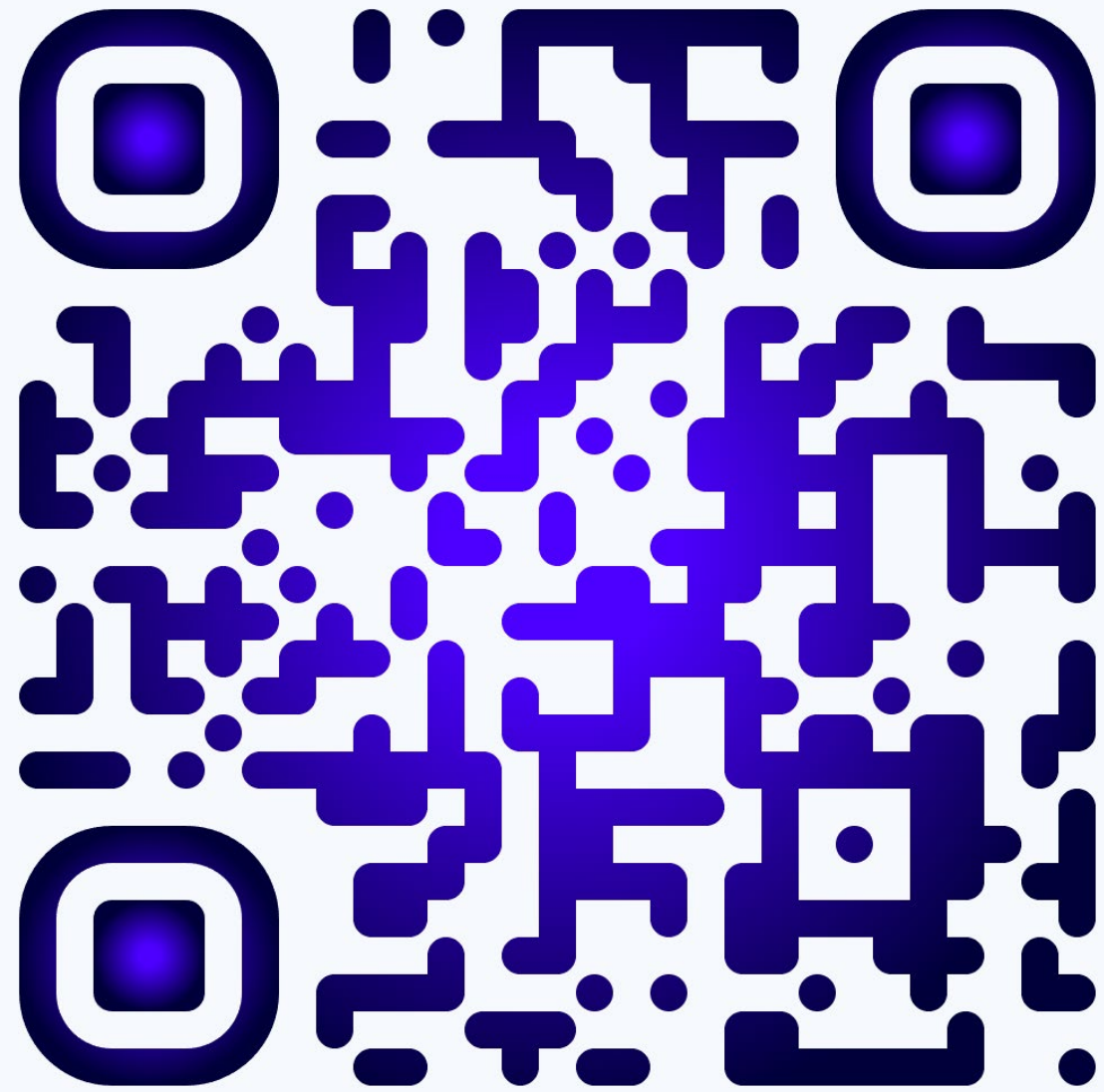
Submit the project, make sure builds are working, then keep going! Add team members and make sure to back up your server.

Do this when nothing is on fire. The point is to learn early while the mistakes cost less.

Resources to get you started



Resources From This Talk



jase.town/ggj-2026

Free tools

Open-source tools to enhance P4

Links to Documentation

Official downloads and Quickstart Guides

P4 Build Machine Lite

A tool to jumpstart your automated builds

P4Godot Plugin

Brand new open-source plugin for Godot

Reference p4ignore files and typemaps

Make setting up your server easier

Thank You

Jase Lindgren

Principal User Advocate, P4

Perforce Software